HDOC Day-53 Activity

Product of Array Except Self Using Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-53

Aim

To write a C program that creates an output array in which each element is the product of all elements of the input array except the element at the same index. The array is passed to **product_except_self()** using its address and division is not used.

Problem Requirements

- Read the size and elements of a variable-length integer array **nums[]**.
- Pass the array to **product_except_self()** using its address.
- Create an output array **answer[]** of the same size.
- Store in **answer[i]** the product of every element except **nums[i]**.
- Do not use division.
- Display the answer elements in their original order, separated by commas.
- The prefix and suffix products fit within a 32-bit integer.

Algorithm

1. Read the size **n** and the elements of **nums[]**.
2. Pass the address of **nums[]** to **product_except_self(nums, n)**.
3. Create an array **answer[n]**.
4. First calculate prefix products. Set **answer[0] = 1**. For every index from 1 to n-1, store the product of all elements to its left.
5. Set **suffix = 1** and traverse the input array from right to left.
6. Multiply **answer[i]** by the current suffix product, then update suffix by multiplying it with **nums[i]**.
7. Print all elements of **answer[]** separated by commas.

C Program

```c
#include <stdio.h>

void product_except_self(int *nums, int n)
{
    int answer[n];

    answer[0] = 1;

    for (int i = 1; i < n; i++)
    {
        answer[i] = answer[i - 1] * (*(nums + i - 1));
    }

    int suffix = 1;

    for (int i = n - 1; i >= 0; i--)
    {
        answer[i] = answer[i] * suffix;
        suffix = suffix * (*(nums + i));
    }

    for (int i = 0; i < n; i++)
    {
        printf("%d", answer[i]);

        if (i < n - 1)
        {
            printf(", ");
        }
    }
}

int main()
{
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int nums[n];

    printf("Enter the elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &nums[i]);
    }

    printf("Product except self: ");
    product_except_self(nums, n);

    return 0;
}
```

Pointer Usage

product_except_self(nums, n) passes the address of the first element of the array. The function parameter **int *nums** receives this address. The expression ***(nums + i)** is equivalent to **nums[i]**.

How the Method Works

The first loop stores the product of all elements to the **left** of each index. The second loop moves from right to left and multiplies this value by the product of all elements to the **right**. Therefore, the current

element itself is never included and division is not required.

Example 1 Dry Run

Index	nums[i]	Left product	Right product	answer[i]
0	1	1	2 x 3 x 4 = 24	24
1	2	1	3 x 4 = 12	12
2	3	1 x 2 = 2	4	8
3	4	1 x 2 x 3 = 6	1	6

Input: 1 2 3 4
Output: 24, 12, 8, 6

Example 2

Input: -1 1 0 -3 3
Output: 0, 0, 9, 0, 0

For the element 0 at index 2, the product of all other elements is (-1) x 1 x (-3) x 3 = **9**. For every other position, the product includes the zero, so the result is 0.

Test Cases / Expected Results

Case	Input Array	Expected Output
Example 1	1 2 3 4	24, 12, 8, 6
Example 2	-1 1 0 -3 3	0, 0, 9, 0, 0

Complexity

Time Complexity: O(n), because the array is traversed a constant number of times.

Space Complexity: O(n) for the required output array **answer[]**. Apart from the output array, only constant extra variables are used.

Result

The program successfully calculates the product of all array elements except the current element for every position. It uses pointers to access the input array, does not use division, correctly handles zero values, and displays the output in the required comma-separated format.

Execution Example

```
Enter the size of array: 4
Enter the elements: 1 2 3 4
Product except self: 24, 12, 8, 6
```